

[21] DNA Sequence Classification with Milvus

저자: Milvus Team

유형: 기술 블로그 포스트, milvus.io

분량: 블로그 포스트 (수 페이지)

역할군: (A) VAQ 동기 부여

요약

이 자료는 전문 벡터 DB 인 Milvus 를 이용하여 DNA 서열 분류를 수행하는 실제 응용 사례를 보여준다. DNA 서열을 벡터로 변환하고 Milvus 에서 유사도 검색을 통해 알려진 서열을 찾아 분류하는 파이프라인을 설명한다. 단순한 벡터 검색뿐 아니라 **종(species), GC 함량, 서열 길이 같은 메타데이터 필터**와 검색을 함께 사용한다는 점이 중요하다. 이는 바이오인포매틱스, 화학, 재료과학 등 **과학 데이터**에서 벡터 DB 의 응용이 확장되고 있으며, 이런 응용에서는 순수 벡터 검색만으로는 부족하고 **필터링·조인·집계 같은 SQL 기능이 필수**임을 보여준다. Exqutor 의 관점에서는 VAQ(Vector-Augmented Queries)가 텍스트/이미지뿐 아니라 과학 데이터까지 응용 범위를 넓히고 있음을 증명하는 사례이다.

상세분석

21.1 DNA 서열과 벡터화의 기초

DNA 데이터의 특성

DNA 는 4 가지 뉴클레오티드(핵염기)로 이루어진 서열이다:

- **A** (아데닌, Adenine)
- **T** (티민, Thymine)
- **G** (구아닌, Guanine)
- **C** (사이토신, Cytosine)

예시 DNA 서열:

```
>genome_123
ATCGATCGATCGATCGTAGCTAGCTAGCTAGC...
```

DNA 서열의 특징:

- 길이: 수백 개 (미토콘드리아 DNA) ~ 수십억 개 (인간 게놈)
- 정보 함량: 각 위치의 뉴클레오티드는 유전 정보를 담음
- 기능적 보존: 진화 과정에서 같은 기능을 하는 서열은 변하지 않음 (보존 영역)

왜 벡터화가 필요한가?

DNA 를 직접 비교하는 것은 매우 비싸다:

Traditional approach:

```
| SEQUENCE 1: ATCGATCG... | (100 만 글자)
| SEQUENCE 2: ATCGAGCG... | (100 만 글자)
|
| 비교 알고리즘: 동적 계획 |
| 시간 복잡도: O(n * m)   |
| = O(100 만 * 100 만)   |
| = 조 번의 연산         |
```

결과: 백만 시퀀스에서 가장 비슷한 것을 찾으려면?

→ 1 조 × 100 만 = 극우주 수준의 연산
→ 실용적이지 않음

벡터화의 해결책:

Vectorization approach:

SEQUENCE 1	
→ k-mer 분석	
→ 벡터로 변환	
→ 벡터 1536D	

비교: 두 1536 차원 벡터 거리 계산

시간 복잡도: $O(1536) = \sim$ 수천 번의 연산

결과: 100 만 시퀀스 검색도 초 단위로 완료

21.2 DNA 서열의 벡터 표현 방법

방법 1: k-mer 기반 특징 벡터

k-mer 의 개념:

DNA 서열: ATCGATCGATCG

k=3 (3-mer 추출):

ATC, TCG, CGA, GAT, ATC, TCG, CGA, ATC, TCG, ATG...

k-mer 빈도 벡터:

전체 가능한 3-mer: $4^3 = 64$ 가지

각 3-mer 의 빈도를 64 차원 벡터로 표현

예시:

AAA: 0, AAC: 0, AAG: 1, AAT: 0,

ACA: 2, ACC: 0, ACG: 1, ACT: 0,

...

TTT: 0, TTC: 1, TTG: 0, TTT: 0

결과 벡터: [0, 0, 1, 0, 2, 0, 1, 0, ..., 0, 1, 0, 0]

특성:

특징	내용
차원	4^k ($k=3 \rightarrow 64$, $k=4 \rightarrow 256$)
생성 속도	빠름 ($O(n)$)
정보 손실	중간 (순서 정보 일부 손실)
용도	빠른 초기 필터링

k 값의 선택:

k=2: 16 차원, 너무 간단 (기능 구분 어려움)

k=3: 64 차원, 일반적 (빠르고 충분한 정보)

k=4: 256 차원, 더 정확 (계산 비용 증가)

k=5+: 고차원, 느림 (실시간 검색 부적절)

방법 2: 사전학습 딥러닝 임베딩 모델

DNA2Vec / DNABERT:

전통적 NLP 모델을 DNA 에 적용:

DNABERT (DNA BERT):

1. DNA 를 "토큰"으로 취급 (k-mer 또는 코돈)
2. 사전학습 언어 모델 (BERT)을 DNA 데이터로 미세조정
3. 각 DNA 서열을 고정 길이 벡터로 변환
4. 유사한 기능의 DNA 는 벡터 공간에서 가까이 위치

장점:

- 생물학적 의미를 포착 (기능 유사성)
- 1536 차원 밀집 벡터 (정보 손실 적음)
- 사전학습으로 일반화 성능 우수

단점:

- 계산 비용 높음 (GPU 필요)
- 블랙박스 모델 (해석 어려움)
- 재학습 비용

방법 3: 하이브리드 방식

DNABERT 임베딩 + k-mer 특징의 하이브리드:

DNABERT: 의미 정보 (기능 유사성)
k-mer: 해석 가능 정보 (서열 특성)

1. DNABERT 로 1536D 벡터 생성
2. k-mer 특징 64D 추가
3. 총 1600D 벡터로 검색

이점:

- 검색 정확도 향상 (두 관점 모두 고려)
- 결과 해석 가능 (k-mer 기여도 확인)

21.3 DNA 분류 파이프라인

기본 구조

1. 쿼리 DNA 서열 입력

예: 새로운 박테리아 서열

↓

2. 벡터 변환

- k-mer 추출

- DNABERT 임베딩

- 결과: 1536D 벡터

↓

3. Milvus 에서 유사 검색

```

SELECT id, species, sequence

FROM dna\_collection

WHERE embedding <-> query\_vec < threshold

LIMIT 10

```

결과: 유사한 서열 10 개

↓

4. 분류 결정

- 검색된 서열의 종(species)을 확인

- 투표 방식: 상위 10 개 중 가장 많은 종

- 신뢰도: 최상위 결과의 거리로 측정

결과: 새로운 서열 = "종 X"로 분류

구체적 코드 예시

```
import milvus
import numpy as np
from sklearn.preprocessing import normalize

# 1. Milvus 연결
client = milvus.Milvus(host='localhost', port=19530)

# 2. DNA 벡터화
from transformers import AutoTokenizer, AutoModel

tokenizer = AutoTokenizer.from_pretrained("dnabert")
model = AutoModel.from_pretrained("dnabert")

def dna_to_vector(sequence):
    # DNABERT 로 임베딩
    inputs = tokenizer(sequence, return_tensors="pt")
    outputs = model(**inputs)
    embedding = outputs.last_hidden_state[:, 0, :].detach().numpy()
    return normalize(embedding)[0] # 정규화

# 3. 쿼리 실행
query_sequence = "ATCGATCGATCGATCGATCGATCG..."
query_vector = dna_to_vector(query_sequence)

# 4. Milvus 에서 검색
results = client.search(
    collection_name='dna_sequences',
    query_records=[query_vector],
    top_k=10,
    params={}
)

# 5. 결과 분석
species_votes = {}
for result in results[0]:
    species = metadata[result.id]['species']
    species_votes[species] = species_votes.get(species, 0) + 1

predicted_species = max(species_votes, key=species_votes.get)
confidence = max(species_votes.values()) / 10 # 상위 10 개 중 비율
```

21.4 실세계 요구사항: 필터링의 필요성

블로그 포스트는 명시적으로 다루지 않지만, 실제 생물정보학 응용에서는 다음과 같은 필터가 필수적이다:

메타데이터 필터의 종류

1. 분류학적 정보

```
WHERE species = 'E. coli'
AND genus = 'Escherichia'
AND kingdom = 'Bacteria'
```

1. 서열 특성

```
WHERE sequence_length BETWEEN 10000 AND 50000
AND gc_content BETWEEN 0.45 AND 0.55
AND has_open_reading_frame = true
```

1. 데이터 출처

```
WHERE source_database = 'NCBI'
AND sequencing_date > '2020-01-01'
AND sequencing_method = 'Illumina'
```

1. 기능 정보

```
WHERE protein_family = 'RecA'
AND organism_status = 'pathogenic'
AND antibiotic_resistance IS NOT NULL
```

복합 필터 쿼리의 예

```
-- 임상에서 흔히 하는 쿼리
SELECT dna_id, species, virulence_score
FROM dna_sequences
WHERE embedding <-> query_vector < 0.5
AND species IN ('Salmonella enterica', 'Vibrio cholerae')
AND gc_content BETWEEN 0.4 AND 0.6
AND sequencing_date > '2023-01-01'
AND antibiotic_resistance & 'ampicillin'
ORDER BY virulence_score DESC
LIMIT 10;
```


이 쿼리는:

- 벡터 검색: `embedding <-> query_vector < 0.5`
- 메타데이터 필터: `species IN (...)`
- 범위 필터: `gc_content BETWEEN 0.4 AND 0.6`
- 시간 필터: `sequencing_date > '2023-01-01'`
- 비트 필터: `antibiotic_resistance & 'ampicillin'`

이것이 VAQ(Vector-Augmented Query)의 실제 모습이다.

21.5 Milvus 의 메타데이터 필터링

Milvus 는 벡터 검색과 메타데이터 필터를 함께 지원한다:

```
# Milvus 스키마 정의
from milvus import FieldSchema, CollectionSchema

fields = [
    FieldSchema(name="id", dtype=DataType.INT64, is_primary=True),
    FieldSchema(name="embedding", dtype=DataType.FLOAT_VECTOR, dim=1536),
    FieldSchema(name="species", dtype=DataType.VARCHAR),
    FieldSchema(name="gc_content", dtype=DataType.FLOAT),
    FieldSchema(name="sequence_length", dtype=DataType.INT64),
    FieldSchema(name="sequencing_date", dtype=DataType.VARCHAR),
]

schema = CollectionSchema(
    fields=fields,
    description="DNA sequences with metadata"
)

client.create_collection(
    collection_name='dna_sequences',
    schema=schema
)

# 벡터 + 메타데이터 함께 삽입
entities = [
    [id_1, id_2, ...], # id
    [vec_1, vec_2, ...], # embedding
    ['E. coli', 'Bacillus', ...], # species
    [0.48, 0.52, ...], # gc_content
    [5000000, 3000000, ...], # sequence_length
    ['2023-01-15', '2023-02-20', ...], # sequencing_date
]

client.insert(
    collection_name='dna_sequences',
    records=entities
)

# 메타데이터 필터와 함께 벡터 검색
expr = "species == 'E. coli' AND gc_content > 0.4"
results = client.search(
    collection_name='dna_sequences',
    query_records=[query_vector],
    top_k=10,
    expr=expr # 메타데이터 필터 조건
)
```

Milvus 의 한계:

- 필터 + 벡터 검색은 지원하지만, 조인은 지원하지 않는다
- 여러 메타데이터 필터는 가능하지만, 다른 벡터 DB 와의 교차 검색은 불가능

21.6 생물정보학 응용의 특성

이 자료가 중요한 이유는 벡터 검색의 응용 범위를 보여주기 때문이다:

기존 응용 (텍스트/이미지)

웹 검색, 추천 시스템, 이미지 검색
 → 사용자가 정의한 "유사성" 개념이 분명함
 → 벡터 검색이 주요 기능

과학 데이터 응용 (DNA, 단백질, 분자)

서열 분류, 구조 검색, 물성 예측
 → 도메인 전문 지식 + 벡터 검색이 결합됨
 → 필터링, 조인, 집계가 필수
 → VAQ 가 필수

생물정보학 데이터의 규모:

인간 게놈:

- 크기: ~3 십억 뉴클레오타이드
- 유사도 검색 시간: 순차 검색으로 분 단위
- 벡터 검색으로: 밀리초 단위 가능

단백질 서열 데이터베이스 (UniProt):

- ~2 억 개의 단백질 서열
- k-mer 필터링 + 벡터 검색 → 실시간 가능

미생물 메타게놈:

- 환경 샘플에서 수백만 개의 서열 추출
- 종 분류, 기능 주석을 동시에 수행

21.7 본 논문과의 관계

이 자료는 VAQ의 동기를 보여주는 **응용 사례 연구**이다:

VAQ가 필요한 이유:

1. 단일 벡터 검색 부족

```
-- 불충분한 쿼리
SELECT * FROM dna_db
WHERE embedding <-> query_vec < 0.5

-- 현실 요구사항
SELECT * FROM dna_db
WHERE embedding <-> query_vec < 0.5
  AND species = target_species
  AND gc_content > 0.4
  AND sequencing_quality > 50
ORDER BY similarity DESC
```

1. 필터 + 벡터 결합

- Milvus: 가능하지만 조인 없음
- PostgreSQL + pgvector: 조인은 가능하지만 카디널리티 추정 불확실
- **Exqutor**: 조인 + 정확한 카디널리티 추정

1. 멀티 테이블 시나리오

```
SELECT s.species, s.virulence, COUNT(*) AS cnt
FROM dna_sequences s
JOIN clinical_cases c ON s.id = c.dna_id
WHERE s.embedding <-> query_vec < 0.5
  AND c.location = 'Hospital-X'
  AND c.isolation_date > '2023-01-01'
GROUP BY s.species, s.virulence
ORDER BY cnt DESC
```

이 쿼리를 효율적으로 처리하려면:

- 벡터 필터 선택도 정확 추정
- 조인 순서 최적화
- 필터와 조인의 통합 계획

추가 제기 문제

1. 임베딩 품질의 검증 부족

DNABERT 로 생성한 벡터가 **기능적 유사성**을 실제로 포착하는지 검증 필요:

- 같은 기능의 단백질 유전자 → 벡터도 가까운가?
- 기능이 다른 단백질 → 벡터도 먼가?
- 진화적 거리 vs 벡터 거리의 상관성?

논문/블로그는 이런 검증을 명시적으로 제시하지 않음.

2. 대규모 데이터에서의 성능

예시는 상대적으로 작은 데이터셋(수백만 서열)을 가정:

- 전체 NCBI 데이터베이스(십억 서열) 규모에서는?
- 실시간 인덱스 업데이트 (매일 수백만 서열 추가) 가능한가?
- Milvus 의 확장성 한계가 있을 수 있음

3. 메타데이터 필터의 복잡성

블로그는 단순 필터(카테고리, 범위)만 다룸:

- 복잡한 논리 (AND/OR/NOT 의 중첩)
- 다중 속성의 정규화 (예: 종의 계층 분류)
- 동적 메타데이터 (시간에 따라 변하는 임상 정보)

이런 경우 SQL 의 SELECT 문이 훨씬 자연스러움.

추가 제기 문제

1. 벡터 표현의 도메인 의존성

DNA 임베딩(DNABERT)이 모든 분류 작업에 최적인가?

- 종 분류: 매우 효과적
- 기능 분류: 중간 수준
- 구조 예측: 별도의 모델 필요

각 응용마다 최적 임베딩 모델이 다를 수 있으므로, "범용 벡터" 개념이 생물정보학에서는 위험.

2. 필터링 조건의 통합 최적화

메타데이터 필터를 벡터 검색과 함께 사용할 때:

전략 1: 벡터 먼저

1. 벡터 검색으로 후보 1000 개 추출
2. 그 중 메타데이터 필터 적용

문제: 필터 선택도 낮으면 유효 결과 부족

전략 2: 필터 먼저

1. 메타데이터 필터로 데이터 축소
2. 축소된 데이터에서 벡터 검색

문제: 필터 선택도 낮으면 인덱스 활용 안 됨

전략 3: 동시 처리 (NHQ 의 아이디어)

필터와 벡터를 그래프에서 동시에 고려

문제: Milvus 는 미지원, SQL 기반 DB 필요

3. 실시간 업데이트의 비용

생물정보학 데이터베이스는 지속적으로 업데이트됨:

- NCBI GenBank: 매일 수백만 서열 추가
- 각 추가 시마다:
 - a) 임베딩 생성 (GPU 사용)
 - b) Milvus 에 삽입
 - c) 인덱스 재구축

이 비용이 검색 성능에 미치는 영향을 측정해야 함.
